

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA17

COURSE OUTCOME COVERED

***CO2:** Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 50%

Duration: 1 Hour

QUESTIONS: 5 Total Marks: 50

Annexure A

Scenario Based Assignment

Code of Conduct:

I _____V.Lasya(192524186)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: v.lasya

Scenario Based Assignment

1. AI-Powered Emergency Drone Routing (Greedy Best-First Search vs A*)

A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic — roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: a heuristic estimate (straight-line distance), partial real-time updates about blocked paths, and variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty.
- ii. Explain how A* would handle the same situation, including how it balances cost and heuristic.
- iii. Analyze the impact of heuristic accuracy on both algorithms.
- iv. Discuss how dynamic changes (blocked paths) affect search performance.
- v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety.

ANSWER:

i. Behaviour of Greedy Best-First Search:

- Greedy Best-First Search selects the next node purely based on the heuristic value $h(n)$ — the straight-line distance to the destination — ignoring the actual cost already spent ($g(n)$).
- Strength: it makes very fast decisions, which suits the drone's urgent need for minimum delay, since only one comparison (heuristic) is needed per step.
- Weakness: because it ignores incurred cost and real terrain/weather cost, it can choose a route that looks close in straight-line distance but is actually blocked, longer, or more hazardous once real-time updates and variable terrain costs are considered.

ii. Behaviour of A* Search:

- A* evaluates nodes using $f(n) = g(n) + h(n)$, combining the actual accumulated cost (weather/terrain-adjusted travel cost so far) with the heuristic estimate of remaining distance.

- This balance means A* will avoid a route that appears close by straight-line distance but has high real travel cost (e.g., through bad weather or difficult terrain), producing a route that is both efficient and cost-aware.
- When the heuristic is admissible, A* guarantees the optimal (minimum-cost) path, which is valuable when both delay and risk must be minimized.

iii. Impact of Heuristic Accuracy:

- For Greedy Best-First Search, heuristic accuracy determines the entire decision — an inaccurate heuristic can lead the drone into unsafe or inefficient routes, since cost is never checked.
- For A*, an admissible and accurate heuristic keeps the search efficient (fewer nodes expanded) while still guaranteeing optimality; an inadmissible (overestimating) heuristic breaks the optimality guarantee, and an underestimating heuristic increases computation without breaking optimality.

iv. Effect of Dynamic Changes (Blocked Paths):

- Both algorithms plan on a snapshot of the environment; when a path becomes blocked mid-flight, the previously computed route becomes invalid and re-planning is required.
- Greedy Best-First Search re-plans quickly but may again select an unsafe shortcut since it still ignores real cost.
- A* re-planning is more computationally expensive but remains cost-aware; incremental/real-time variants such as D* Lite or Anytime Repairing A* are commonly used to update the route efficiently as blockages are reported, without restarting the search from scratch.

v. Recommendation:

- A* (or an incremental real-time variant such as D* Lite) is the more suitable choice, because it balances safety (avoids costly/hazardous paths that only look attractive by straight-line distance) with near-optimal routing.
- To also satisfy the strict real-time constraint, A* should be combined with a well-designed admissible heuristic and incremental re-planning, giving both fast reaction to blocked paths and dependable, low-risk route selection — better overall than pure Greedy search, which sacrifices safety and optimality for speed alone.

2. Smart City Traffic Signal Optimization (Hill Climbing, Simulated Annealing, Genetic Algorithms)

A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize total waiting time, fuel consumption, and traffic congestion. The system must continuously adjust signals based on traffic flow. However, the search space is extremely large, traffic patterns change dynamically, and the system may get stuck in locally optimal solutions.

Tasks: Formulate this as an optimization problem and explain why traditional search is inefficient.

- i. Explain how Hill Climbing would work in this scenario and identify its limitations.
- ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation.
- iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations.
- iv. Recommend the best approach and justify your choice based on scalability and adaptability.

ANSWER:

Problem Formulation:

- This is an optimization problem, not a goal-directed search problem: the state is the set of signal timing values for every intersection, and the objective function is a weighted combination of total waiting time, fuel consumption, and congestion, to be minimized.
- Traditional goal-based search (BFS/DFS/A*) is inefficient here because there is no single defined goal state to reach — the aim is to continuously improve a numeric objective over an extremely large, continuous, and constantly changing state space (hundreds of intersections $\times$ many possible timings), which is better suited to optimization/local-search methods.

i. Hill Climbing Behaviour and Limitations:

- Hill Climbing starts with an initial timing configuration and repeatedly makes small adjustments (e.g., increase/decrease green time at one intersection), accepting the change only if it reduces waiting time/congestion.
- Limitation: it only accepts improving moves, so it easily gets stuck in a locally optimal timing plan and cannot escape once no single small change improves the objective, even if a better global configuration exists.

ii. Local Maxima, Plateaus, and Ridges:

- Local maxima: a signal-timing configuration where every small change makes things worse locally, even though a globally better city-wide configuration exists — e.g., optimizing one busy intersection while worsening the overall network flow.

- Plateaus: regions where many neighboring timing configurations produce nearly identical waiting times, so the algorithm cannot tell which direction leads to improvement — common when traffic is evenly distributed and small timing changes have negligible effect.
- Ridges: improvement requires simultaneous coordinated changes across multiple intersections (e.g., a 'green wave'), but Hill Climbing changes one variable at a time and cannot find the combined move needed to progress along the ridge.

iii. How Simulated Annealing / Genetic Algorithms Help:

- Simulated Annealing occasionally accepts a worse configuration with a probability that decreases over time (cooling schedule), allowing it to escape local maxima and plateaus that would trap Hill Climbing.
- Genetic Algorithms maintain a population of candidate signal-timing configurations and use selection, crossover, and mutation across generations, allowing coordinated multi-intersection changes (addressing ridges) and parallel exploration of many regions of the search space simultaneously.

iv. Recommendation:

- A Genetic Algorithm (optionally hybridized with Simulated Annealing for local fine-tuning) is recommended, since it scales well to hundreds of intersections through parallel population-based search and adapts naturally to dynamically changing traffic by periodically re-evaluating and evolving the population as new traffic data arrives.

3. Autonomous Mars Rover Exploration (Online Search Agents)

An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability, and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

- i. Explain why this problem requires an online search agent instead of offline search.
- ii. Analyze the challenges of partial observability and dynamic environments.
- iii. Describe how the rover builds and updates its internal model.
- iv. Compare online search with uninformed and informed search strategies.
- v. Justify the most suitable approach considering uncertainty, adaptability, and safety.

ANSWER:

i. Why an Online Search Agent is Required:

- Offline search assumes the entire environment/map is known in advance so a full solution can be computed before acting; the rover has no such prior map of the Martian terrain.
- An online search agent interleaves computation with action — it takes an action, observes the actual outcome/new terrain, and then decides the next action — which matches the rover's need to explore and learn about hazards as it moves.

ii. Challenges of Partial Observability and Dynamic Environments:

- Partial observability: the rover's sensors only reveal a limited area around it, so hazards outside sensor range (craters, rocks) remain unknown until the rover gets closer, risking unsafe moves.
- Dynamic environment: terrain conditions can change over time (dust storms, small rockslides), so even previously observed information may become outdated, requiring the rover to continuously re-verify and re-plan rather than trust a static map.

iii. Building and Updating the Internal Model:

- The rover incrementally builds a local map/graph from sensor data (cameras, LIDAR), marking cells as traversable, hazardous, or unknown as it explores.
- Cost and heuristic estimates for unexplored/explored regions are updated based on actual experience — an approach similar to Learning Real-Time A* (LRTA*), where the rover updates its heuristic values after each move using the true cost it just observed.

iv. Online Search vs Uninformed and Informed (Offline) Search:

- Uninformed search (e.g., BFS/DFS) explores blindly without any heuristic guidance and assumes a known state space — highly inefficient and unrealistic for an unmapped planet.
- Informed offline search (e.g., A*) requires a complete map to compute the optimal path in advance, which is not available here.
- Online search agents act step-by-step using available (heuristic) information plus real sensor feedback, trading a strict optimality guarantee for the ability to function safely and adaptively under incomplete knowledge.

v. Justification of the Most Suitable Approach:

- An online, informed search approach (e.g., LRTA*-style learning real-time search) is most suitable, since it allows the rover to make locally informed decisions using heuristic guidance, continuously update its knowledge as new terrain is sensed, and avoid known hazards — directly addressing the uncertainty, adaptability, and safety requirements of Mars exploration, which pure offline or uninformed search cannot provide.

4. University Exam Timetable Generation (Constraint Satisfaction Problem)

A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: no student should have overlapping exams, limited number of exam halls, certain subjects must be scheduled before others, faculty availability constraints, and special accommodations for some students. The complexity increases as the number of courses and students grows.

- Clearly define variables, domains, and constraints with explanation.
- Explain how backtracking search works in this scenario.
- Discuss how constraint propagation (e.g., forward checking) improves efficiency.
- Analyze real-world challenges and justify the best strategy for scalability.

ANSWER:

CSP Formulation:

- Variables: each exam/course to be scheduled (one variable per subject).
- Domains: the set of valid (time-slot, exam-hall) pairs available for that subject, based on hall capacity and availability.
- Constraints: (a) no two exams sharing a common student may be assigned the same time slot; (b) each hall can host only one exam per time slot and must have sufficient capacity; (c) precedence constraints — certain subjects must be scheduled before others; (d) faculty availability — the invigilating faculty must be free at the assigned slot; (e) special accommodation constraints — some students require a separate hall or extra time, linking their exam to specific slot/hall options.

i. Variables, Domains, and Constraints (Explained):

- Defining exams as variables with (time-slot, hall) domains turns the scheduling problem into a classic CSP: a valid solution is any assignment of domain values to all variables that satisfies every constraint simultaneously (no overlaps, capacity respected, precedence honored, faculty free, accommodations met).

ii. How Backtracking Search Works Here:

- Backtracking assigns a time-slot/hall to one exam at a time, checking after each assignment whether it violates any constraint against previously assigned exams (e.g., creates a student overlap or exceeds hall capacity).
- If a violation occurs, the search backtracks and tries the next available domain value for that exam; this continues recursively until a complete, consistent timetable is found or the search proves no solution exists with the current setup.

iii. How Constraint Propagation (Forward Checking) Improves Efficiency:

- Forward checking removes now-inconsistent values from the domains of unassigned exams immediately after each assignment — e.g., once an exam is given a slot, that slot is removed from the domains of all exams sharing a student with it.
- This detects failures earlier (an empty domain signals an unsolvable branch immediately) rather than discovering the conflict much later, significantly reducing the number of backtracks and speeding up the search as the number of courses/students grows.

iv. Real-World Challenges and Recommended Strategy:

- As courses and students increase, the constraint graph becomes dense (many overlapping student registrations), hall/faculty availability becomes tighter, and pure backtracking alone becomes slow.
- A scalable strategy combines backtracking with forward checking (or full arc-consistency, e.g., AC-3) and variable-ordering heuristics such as Minimum Remaining Values (MRV) and the Degree heuristic to tackle the most constrained exams first, and value-ordering heuristics (least constraining value) to reduce future conflicts.
- For very large universities, a local-search repair method such as min-conflicts can be used to quickly resolve remaining conflicts in an initially complete but imperfect assignment, making the overall approach practical at scale.

5. Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning)

A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, make decisions within strict time limits, and evaluate complex game states with multiple possible moves. The decision tree is extremely large, making computation expensive.

- i. Explain how the Minimax algorithm models this problem.
- ii. Analyze the computational challenges of large game trees.
- iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning.
- iv. Design an evaluation function and justify the factors considered.
- v. Recommend a strategy for balancing optimality and real-time performance.

ANSWER:

i. How Minimax Models the Problem:

- Minimax represents the game as a tree of alternating turns: the AI (maximizing player) tries to choose moves that maximize its outcome, while assuming the human opponent (minimizing player) will always choose moves that minimize the AI's outcome.
- Values are computed recursively from the leaves (or a cutoff depth) back up to the root, so the AI's chosen move is the one that yields the best guaranteed result assuming optimal opposition.

ii. Computational Challenges of Large Game Trees:

- The number of nodes in the game tree grows exponentially as $O(b^d)$, where b is the branching factor (number of possible moves) and d is the search depth; real-time strategy games typically have a very large branching factor.
- This makes full-depth Minimax computationally infeasible within the strict time limits of real-time play, and also memory-intensive if states/paths need to be stored.

iii. How Alpha-Beta Pruning Improves Efficiency:

- Alpha-Beta pruning tracks two bounds while traversing the tree — alpha (the best value the maximizer can guarantee so far) and beta (the best value the minimizer can guarantee so far).
- Whenever a branch is found where $\beta \leq \alpha$, that branch cannot possibly affect the final decision at the root and is pruned (skipped) without being fully evaluated.
- This does not change the final Minimax decision but drastically reduces the number of nodes evaluated — with ideal move ordering (best moves examined first), it reduces the complexity from $O(b^d)$ to $O(b^{(d/2)})$, effectively allowing the AI to search twice as deep in the same time budget.

iv. Evaluation Function Design:

- Since the full tree cannot be searched to terminal states within time limits, the search is cut off at a fixed depth and an evaluation function estimates the desirability of that non-terminal state.

- Suggested factors: material/resource advantage (units, health, resources owned), positional control (territory/map control), mobility (number of available future moves/options), threat level to key assets (base, key units), and tactical opportunities (possible attacks or captures) — combined as a weighted sum, with weights tuned through testing or learning.

v. Strategy for Balancing Optimality and Real-Time Performance:

- Use depth-limited Minimax with Alpha-Beta pruning combined with iterative deepening — searching depth 1, then 2, then 3, and so on within the time budget — so a valid best move is always ready even if time runs out mid-search.
- Combine this with strong move-ordering heuristics (trying likely-best moves first) to maximize the benefit of Alpha-Beta pruning, and a well-tuned evaluation function, giving the AI near-optimal decisions while reliably respecting the strict real-time constraints of the game.

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand